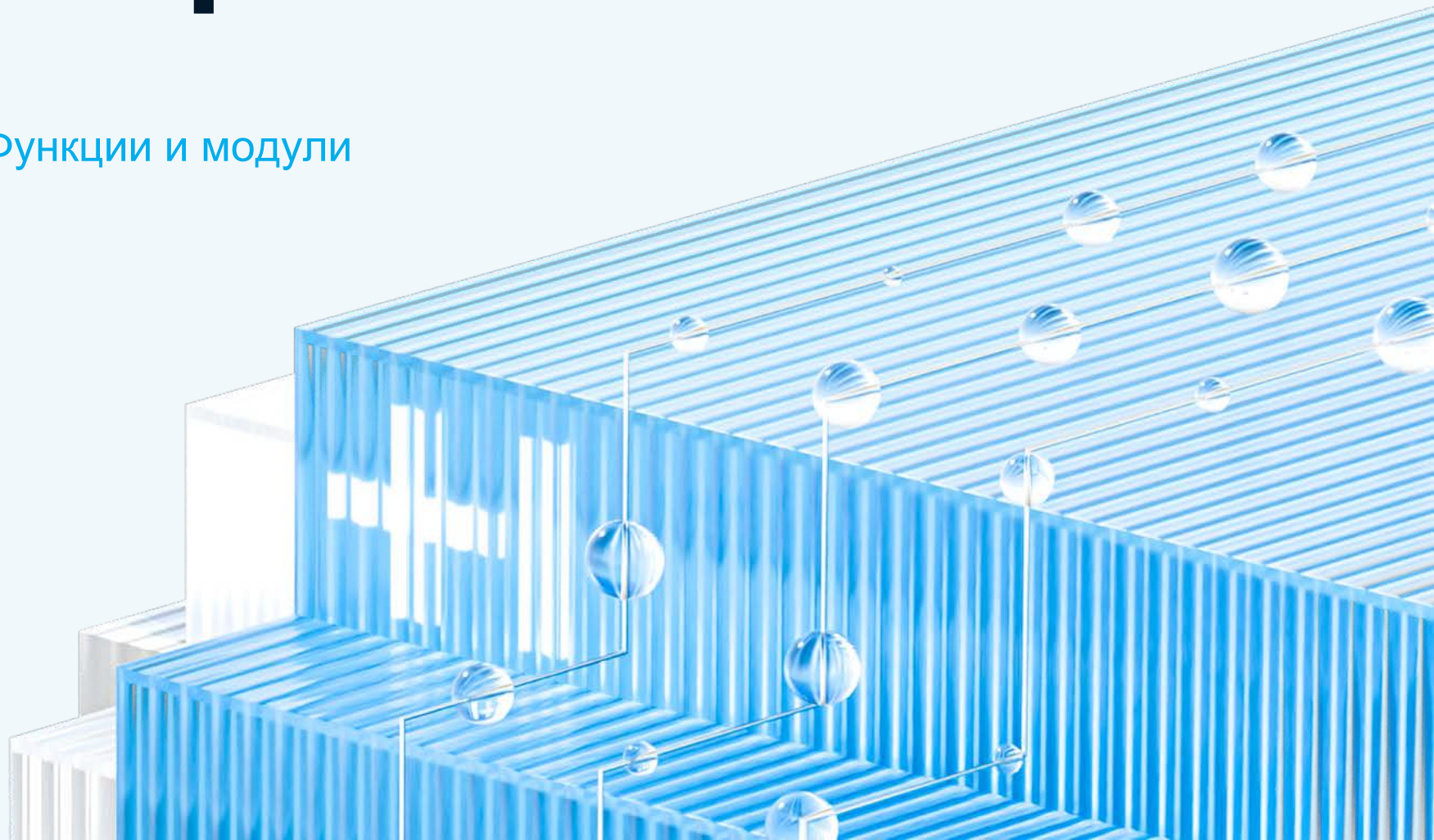
Основы программирования

05.11.2026

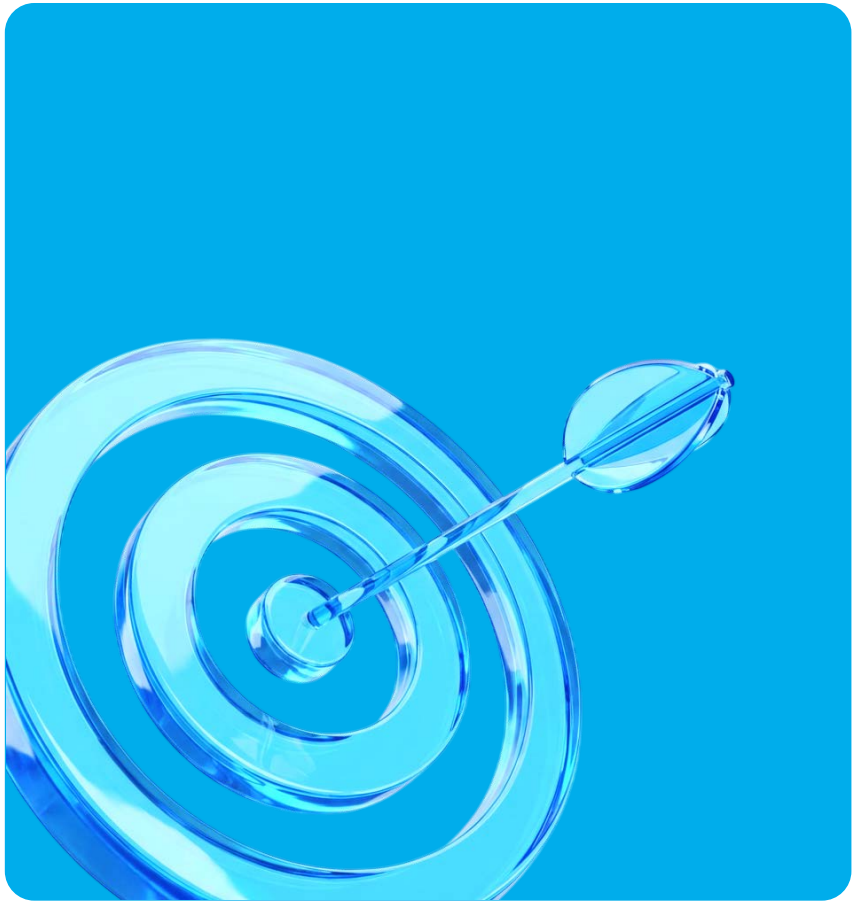
Лабораторная работа № 8. Функции и модули



Проклов Роман Сергеевич

ITI

Цели лабораторной работы



1. **Познакомиться** с понятием функции, рекурсии и модуля
2. **Научиться** объявлять и вызывать функции в Python и C++
3. **Изучить** передачу параметров, возврат значений, области видимости
4. **Освоить** рекурсивные алгоритмы, создание собственных модулей
5. **Применить** полученные знания для модульного программирования
6. **Отработать** навыки работы с Git: коммиты, ветвление, push

Формируемые компетенции: ППК-Р1.1, ППК-Р1.4, ППК-Р1.5

План лабораторной работы

Часть 1. Теория: Функции

1. Что такое функция? Зачем нужны функции?
2. Объявление и вызов функций в Python и C++
3. Параметры и возврат значений
4. Области видимости
5. Перегрузка функций (C++)
6. Лямбда-выражения

Часть 2. Теория: Рекурсия

7. Что такое рекурсия? Базовый случай
8. Примеры рекурсивных алгоритмов
9. Сравнение рекурсии и итерации

Часть 3. Теория: Модули

10. Что такое модуль? Зачем нужны модули?
11. Создание собственных модулей в Python
12. Создание собственных модулей в C++
13. Обработка исключений в модулях

Часть 4. Практика

14. Задания на закрепление + Работа в мини-группах
15. Сдача отчёта и защита



Функции

Функция: именованный блок кода

Функция — это именованный блок кода, который выполняет определённую задачу и может быть вызван из разных мест программы.

Зачем нужны функции?

- **Избежать дублирования кода:** пишем один раз — используем многократно
- **Разбить задачу на подзадачи:** каждая функция решает одну задачу
- **Упростить тестирование:** функции легко тестировать отдельно
- **Улучшить читаемость:** код становится понятнее
- **Переиспользовать код:** функцию можно использовать в других программах

Аналогия из жизни:

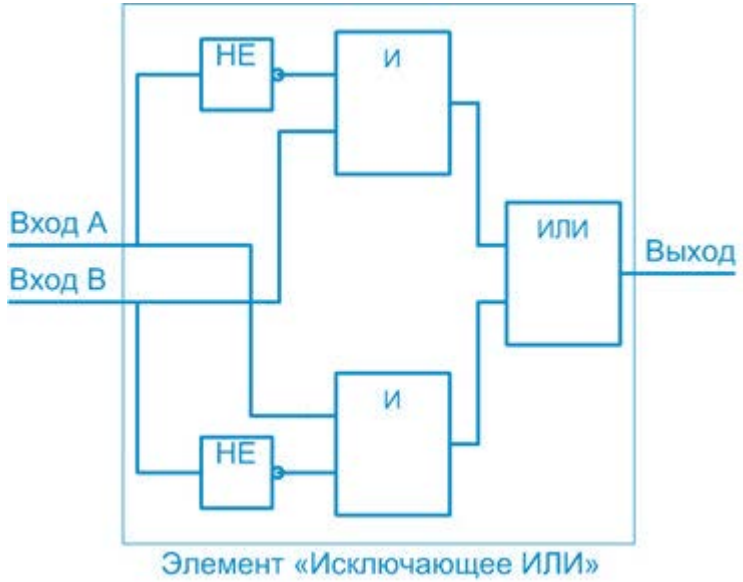
- **Функция** — это как кнопка на пульте. Нажимаем — что-то происходит
- **Параметры** — это как настройки: громкость, канал
- **Возвращаемое значение** — это результат

Структура функции:

```
[имя функции]([параметры]) → [возвращаемое значение]
```

Роль в программировании:

Функции — это «кирпичики», из которых строится программа.



Элемент «Исключающее ИЛИ»

Функции в Python: объявление и вызов

Синтаксис:

```
def имя_функции(параметры):
    """Docstring: описание функции."""
    # тело функции
    return результат
```

Пример: простейшая функция

```
def greet():
    """Выводит приветствие."""
    print("Привет, мир!")

greet()     # Привет, мир!
```

Пример: функция с параметрами

```
def greet(name):
    """Приветствует пользователя по имени."""
    print(f"Привет, {name}!")

greet("Иван")          # Привет, Иван!
greet("Мария")         # Привет, Мария!
```

Пример: функция с возвратом значения

```
def square(x):
    """Возвращает квадрат числа."""
    return x * x

result = square(5)
print(result)              # 25
```

Аннотации типов:

```
def add(a: int, b: int) -> int:
    """Складывает два числа."""
return a + b
```

Функции в C++: объявление и вызов

Синтаксис:

```
тип_возврата имя_функции(параметры) {
    // тело функции
    return результат;
}
```

Пример: функция с параметрами

```
#include <iostream> #include <string>

void greet(const std::string& name) {
    std::cout << "Привет, " << name << "!\n";
}

int main() {
    greet("Иван");
    greet("Мария");
    return 0;
}
```

Пример: простейшая функция

```
#include <iostream>

void greet() {
    std::cout << "Привет, мир!\n";
}

int main() {
    greet();
    return 0;
}
```

Прототип функции:

```
int square(int x);        // прототип

int main() {
    std::cout << square(5) << "\n";
    return 0;
}

int square(int x) {       // определение
    return x * x;
}
```

Пример: функция с возвратом значения

```
int square(int x) {
    return x * x;
}

int main() {
    int result = square(5);
    std::cout << result << "\n";      // 25
    return 0;
}
```

Python vs C++: сравнение функций

Аспект	Python	C++
Ключевое слово	def	Нет (тип возврата)
Тип возврата	Не указывается (или аннотация)	Обязателен
Типы параметров	Не указываются (или аннотации)	Обязательны
Docstring	Да	Комментарии
Прототип	Не нужен	Нужен (иногда)
Отступы	Обязательны	Для читаемости
Точка с запятой	Не нужна	Обязательна

Пример: функция вычисления факториала

Python:

```python
def factorial(n: int) -> int:
    """Вычисляет факториал числа n."""
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print(factorial(5))        # 120
```

C++:

```cpp
#include <iostream>

int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; ++i) {
        result *= i;
    }
    return result;
}

int main() {
    std::cout << factorial(5) << "\n";     // 120
    return 0;
}
```

Передача параметров в Python

Python — передача по ссылке на объект:

Изменяемые объекты (list, dict, set):

```
def add_item(lst, item):
    """Добавляет элемент в список."""
    lst.append(item)

my_list = [1, 2, 3]
add_item(my_list, 4)
print(my_list)          # [1, 2, 3, 4] — список изменён!
```

Неизменяемые объекты (int, str, tuple):

```
def increment(x):
    """Увеличивает число на 1."""
    x += 1

number = 5
increment(number)
print(number)          # 5 — число не изменено!
```

Параметры по умолчанию:

```
def greet(name, greeting="Привет"):
    print(f"{greeting}, {name}!")

greet("Иван")                    # Привет, Иван!
greet("Мария", "Здравствуй")     # Здравствуй, Мария!
```

Произвольное число аргументов:

```
def sum_all(*args):
    return sum(args)

print(sum_all(1, 2, 3))                # 6

def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Иван", age=25)
```

Именованные аргументы:

```
def create_profile(name, age, city):
    print(f"{name}, {age} лет, из {city}")

create_profile(age=25, name="Иван", city="Москва")
```

Передача параметров в C++

C++ — три способа передачи:

1. По значению (копия):

```
void increment(int x) {
    x += 1;     // изменяется только копия
}

int main() {
    int number = 5;
    increment(number);
    std::cout << number << "\n"; // 5 — не изменено!
    return 0;
}
```

2. По ссылке (&):

```
void increment(int& x) {
    x += 1;     // изменяется оригинал
}

int main() {
    int number = 5;
    increment(number);
    std::cout << number << "\n";     // 6 — изменено!
    return 0;
}
```

3. По константной ссылке (const &):

```
void printName(const std::string& name) {
    // name += "!"; // ОШИБКА! Нельзя изменять
    std::cout << name << "\n";
}
```

Параметры по умолчанию:

```
void greet(const std::string& name, const std::string& greeting = "Привет") {
    std::cout << greeting << ", " << name << "!\n";
}
```

Когда что использовать:

- **По значению:** для маленьких типов (int, double, char)
- **По ссылке:** когда нужно изменить аргумент
- **По константной ссылке:** для больших объектов (string, vector)

Возврат значений: Python vs C++

Python — возврат одного значения:

```python
def square(x):
    return x * x
```

Python — возврат нескольких значений (кортеж):

```python
def min_max(numbers):
    """Возвращает минимум и максимум."""
    return min(numbers), max(numbers)

minimum, maximum = min_max([3, 1, 4, 1, 5])
```

C++ — возврат одного значения:

```cpp
int square(int x) {
    return x * x;
}
```

C++ — возврат нескольких значений:

```cpp
#include <utility>

std::pair<int, int> minMax(const std::vector<int>& v) {
    auto [min_it, max_it] = std::minmax_element(v.begin(), v.end());
    return {*min_it, *max_it};
}

int main() {
    std::vector<int> v = {3, 1, 4, 1, 5};
    auto [minimum, maximum] = minMax(v);
    return 0;
}
```

C++ — возврат void:

```cpp
void printGreeting(const std::string& name) {
    std::cout << "Привет, " << name << "!\n";
}
```

Области видимости переменных

Python — правило LEGB:

Уровень	Название	Описание
L	Local	Локальные переменные функции
E	Enclosing	Переменные внешней функции
G	Global	Глобальные переменные модуля
B	Built-in	Встроенные имена

```python
x = 10   # глобальная

def func():
    x = 20 # локальная
    print(x)    # 20

func()
print(x)        # 10
```

Python — global и nonlocal:

```python
counter = 0

def increment():
    global counter
    counter += 1
```

Совет: избегайте глобальных переменных.

C++ — области видимости:

```cpp
int x = 10;                 // глобальная

void func() {
    int x = 20;             // локальная
    std::cout << x << "\n";         // 20
}
```

C++ — блочная область:

```cpp
int main() {
    int x = 10; {
        int x = 20;     // другая переменная
        std::cout << x << "\n";     // 20
    }
    std::cout << x << "\n";         // 10
}
```

Перегрузка функций в C++

Перегрузка функций — это возможность иметь несколько функций с одинаковым именем, но разными параметрами.

Правила перегрузки:

- Функции различаются по типам или количеству параметров
- Нельзя перегружать по типу возврата
- Компилятор выбирает подходящую функцию по аргументам

В Python перегрузки нет:

Python не поддерживает перегрузку функций. Используются:

- Параметры по умолчанию
- *args и **kwargs
- Проверка типов внутри функции

Пример:

```
#include <iostream> #include <string>

void print(int x) {
    std::cout << "Целое: " << x << "\n";
}

void print(double x) {
    std::cout << "Дробное: " << x << "\n";
}

void print(const std::string& s) {
    std::cout << "Строка: " << s << "\n";
}

int add(int a, int b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c; }

int main() {
    print(5);                // Целое: 5
    print(3.14);             // Дробное: 3.14
    print("Привет");         // Строка: Привет

    std::cout << add(2, 3) << "\n";                    // 5
    std::cout << add(2, 3, 4) << "\n";    // 9

    return 0;
}
```

Лямбда-выражения в Python

Лямбда-выражение — это анонимная функция, которая может быть определена в одной строке.

Синтаксис:

```
lambda параметры: выражение
```

Примеры:

```
# Обычная функция
def square(x):
    return x * x

# Лямбда-эквивалент
square_lambda = lambda x: x * x

print(square(5))                    # 25
print(square_lambda(5))             # 25

# Лямбда с несколькими параметрами
add = lambda a, b: a + b
print(add(3, 5))          # 8

# Лямбда с условием max_value = lambda a, b: a if a > b else b
print(max_value(10, 20))            # 20
```

Применение лямбд:

```
# Сортировка с ключом
students = [("Иван", 25), ("Мария", 22), ("Пётр", 30)]
students.sort(key=lambda s: s[1])

# Фильтрация
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))

# Преобразование
squares = list(map(lambda x: x ** 2, numbers))

# С reduce
from functools import reduce
product = reduce(lambda x, y: x * y, [1, 2, 3, 4, 5])
```

Ограничения лямбд:

- Только одно выражение
- Нельзя использовать return
- Нельзя использовать сложные конструкции

Лямбда-выражения в C++ (C++11+)

Захват переменных:

```
int x = 10;
int y = 20;

// Захват по значению
auto byValue = [x, y]() { return x + y; };

// Захват по ссылке
auto byRef = [&x, &y]() { x += 1; y += 1; };

// Захват всего по значению
auto allValue = [=]() { return x + y; };

// Захват всего по ссылке
auto allRef = [&]() { x += 1; };
```

Синтаксис:

```
[захват](параметры) -> тип_возврата {
    // тело
}
```

Примеры:

```
#include <iostream>

int main() {
    // Простая лямбда
    auto square = [](int x) { return x * x; };
    std::cout << square(5) << "\n";                // 25

    // Лямбда с несколькими параметрами
    auto add = [](int a, int b) { return a + b; };
    std::cout << add(3, 5) << "\n";                // 8

    return 0;
}
```

Применение в алгоритмах:

```
#include <vector> #include <algorithm>

std::vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Сортировка по убыванию
std::sort(numbers.begin(), numbers.end(), [](int a, int b) { return a > b; });

// Фильтрация
std::vector<int> evens;
std::copy_if(numbers.begin(), numbers.end(), std::back_inserter(evens), [](int x) { return x % 2 == 0; });

// Преобразование
std::vector<int> squares(numbers.size());
std::transform(numbers.begin(), numbers.end(), squares.begin(), [](int x) { return x * x; });
```

Рекурсия

Рекурсия: функция, вызывающая себя

Рекурсия — это приём, когда функция вызывает саму себя.

Обязательные компоненты:

1. **Базовый случай** — условие остановки
2. **Рекурсивный шаг** — вызов функции с изменённым аргументом

Аналогия из жизни:

- Матрёшка: открываем — внутри ещё одна матрёшка
- Зеркала: отражение в отражении

Пример: факториал

```
5! = 5 * 4!
4! = 4 * 3!
3! = 3 * 2!
2! = 2 * 1!
1! = 1 (базовый случай)
```

Плюсы и минусы:

Плюсы	Минусы
Элегантность	Риск переполнения стека
Простота для рекурсивных структур	Медленнее итерации
Естественность для деревьев, графов	Сложнее отладка

Совет: всегда проверяйте базовый случай!

Рекурсия в Python

Факториал:

```
def factorial(n):
    """Вычисляет факториал числа n."""
    if n <= 1:
        return 1          # базовый случай
    return n * factorial(n - 1)     # рекурсивный шаг

print(factorial(5))       # 120
```

Числа Фибоначчи:

```
def fib(n):
    """Возвращает n-е число Фибоначчи."""
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(10))            # 55
```

Сумма элементов списка:

```
def sum_list(lst):
    """Рекурсивно суммирует элементы списка."""
    if not lst:
        return 0
    return lst[0] + sum_list(lst[1:])

print(sum_list([1, 2, 3, 4, 5]))       # 15
```

Ханойские башни:

```
def hanoi(n, source, target, auxiliary):
    """Решает задачу Ханойских башен."""
    if n == 1:
        print(f"Переместить диск 1 с {source} на {target}")
        return
    hanoi(n - 1, source, auxiliary, target)
    print(f"Переместить диск {n} с {source} на {target}")
    hanoi(n - 1, auxiliary, target, source)

hanoi(3, 'A', 'C', 'B')
```

Рекурсия в C++

Факториал:

```cpp
#include <iostream>

int factorial(int n) {
    if (n <= 1) {
        return 1;        // базовый случай
    }
    return n * factorial(n - 1);        // рекурсивный шаг
}

int main() {
    std::cout << factorial(5) << "\n";              // 120
    return 0;
}
```

Сумма элементов вектора:

```cpp
#include <vector>

int sumVector(const std::vector<int>& v, size_t index = 0) {
    if (index >= v.size()) {
        return 0;
    }
    return v[index] + sumVector(v, index + 1);
}
```

Числа Фибоначчи:

```cpp
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Ханойские башни:

```cpp
void hanoi(int n, char source, char target, char auxiliary) {
    if (n == 1) {
        std::cout << "Переместить диск 1 с " << source << " на " << target << "\n";
        return;
    }
    hanoi(n - 1, source, auxiliary, target);
    std::cout << "Переместить диск " << n << " с " << source << " на " << target << "\n";
    hanoi(n - 1, auxiliary, target, source);
}
```

Оптимизация:

- Используйте long long для факториала
- Для Фибоначчи лучше итерация или мемоизация

Рекурсия vs итерация

Аспект	Рекурсия	Итерация
Читаемость	Элегантнее для рекурсивных задач	Проще для линейных задач
Производительность	Медленнее (накладные расходы)	Быстрее
Память	Использует стек вызовов	Константная
Риск	Переполнение стека	Нет
Отладка	Сложнее	Проще
Применение	Деревья, графы, divide-and-conquer	Циклы, линейные задачи

Пример: факториал

Рекурсия:

```
def factorial_rec(n):
    if n <= 1:
        return 1
    return n * factorial_rec(n - 1)
```

Итерация:

```
def factorial_iter(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Когда использовать рекурсию:

- Задача имеет рекурсивную структуру (дерево, граф)
- Решение через рекурсию значительно проще
- Глубина рекурсии невелика

Когда использовать итерацию:

- Линейные задачи
- Важна производительность
- Большая глубина

Модули

Модуль: организация кода

Модуль — это файл, содержащий код (функции, классы, переменные), который можно подключать к другим программам.

Зачем нужны модули?

- **Организация кода:** разбиение на логические части
- **Переиспользование:** один модуль — много программ
- **Избежание конфликтов:** пространства имён
- **Ускорение компиляции:** изменяется только один файл
- **Командная работа:** каждый работает над своим модулем

Структура модуля:

```
project/
├── main.py          # основная программа
├── math_utils.py    # модуль математических функций
├── string_utils.py  # модуль строковых функций
└── file_utils.py    # модуль файловых операций
```

Аналогия из жизни:

- Модуль — это как ящик с инструментами
- Можно использовать в разных проектах
- Можно комбинировать с другими ящиками

Виды модулей:

- Стандартные: math, os, sys, random (Python); <iostream>, <vector> (C++)
- Сторонние: устанавливаются через pip (Python), vcpkg / conan (C++)
- Собственные: создаём сами

Создание модулей в Python

Создание модуля math_utils.py:

```python
"""Модуль математических функций."""

def add(a: float, b: float) -> float:
    """Складывает два числа."""
    return a + b

def subtract(a: float, b: float) -> float:
    """Вычитает b из a."""
    return a - b

def multiply(a: float, b: float) -> float:
    """Умножает два числа."""
    return a * b

def divide(a: float, b: float) -> float:
    """Делит a на b."""
    if b == 0: raise ValueError("Деление на ноль")
    return a / b

# Константы
PI = 3.1415926535
E = 2.7182818284
```

Использование модуля main.py:

```python
"""Основная программа."""
import math_utils
from math_utils import add, subtract

# Через имя модуля
result1 = math_utils.multiply(3, 4)

# Через импорт конкретных функций
result2 = add(3, 5)

print(f"Результат 1: {result1}")     # 12
print(f"Результат 2: {result2}")     # 8
print(f"PI = {math_utils.PI}")
```

Проверка __name__:

```python
# math_utils.py
def add(a, b):
    return a + b

if __name__ == "__main__":
    # Этот код выполняется только при прямом запуске
    print("Тестирование модуля")
    print(add(2, 3))
```

Создание модулей в C++

Заголовочный файл math_utils.h:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

double add(double a, double b);
double subtract(double a, double b);
double multiply(double a, double b);
double divide(double a, double b);

const double PI = 3.1415926535;
const double E = 2.7182818284;

#endif
```

Файл реализации math_utils.cpp:

```
#include "math_utils.h"
#include <stdexcept>

double add(double a, double b) { return a + b; }
double subtract(double a, double b) { return a - b; }
double multiply(double a, double b) { return a * b; }

double divide(double a, double b) {
    if (b == 0) {
        throw std::invalid_argument("Деление на ноль");
    }
    return a / b;
}
```

Основная программа main.cpp:

```
#include <iostream> #include "math_utils.h"

int main() {
    std::cout << add(3, 5) << "\n";              // 8
    std::cout << multiply(3, 4) << "\n";         // 12
    std::cout << "PI = " << PI << "\n";          // 3.14159

    try {
        std::cout << divide(10, 0) << "\n";
    } catch (const std::exception& e) {
        std::cout << "Ошибка: " << e.what() << "\n";
    }
    return 0;
}
```

Компиляция:

```
g++ -std=c++17 -o app main.cpp math_utils.cpp
```

Защита от повторного включения:

- #ifndef ... #define ... #endif — классический способ
- #pragma once — современный способ

Python vs C++: модули

Аспект	Python	C++
Файл	.py	.h + .cpp
Подключение	import module	#include "module.h"
Защита от повторного включения	Не нужна	#ifndef / #pragma once
Пространства имён	namespace (опционально)	namespace (рекомендуется)
Компиляция	Не нужна	Нужна
Точка входа	if __name__ == "__main__":	int main()

Пример: модуль для работы со строками

Python (string_utils.py):

```python
def to_upper(s: str) -> str:
    return s.upper()

def count_words(s: str) -> int:
    return len(s.split())
```

C++ (string_utils.h):

```cpp
#ifndef STRING_UTILS_H #
define STRING_UTILS_H

#include <string>

std::string toUpper(const std::string& s);
int countWords(const std::string& s);

#endif
```

C++ (string_utils.cpp):

```cpp
#include "string_utils.h" #include <algorithm>
#include <sstream> #include <cctype>

std::string toUpper(const std::string& s) {
    std::string result = s;
    std::transform(result.begin(), result.end(), result.begin(),
                   [](unsigned char c) { return std::toupper(c); });
    return result;
}

int countWords(const std::string& s) {
    std::istringstream iss(s);
    std::string word;
    int count = 0;
    while (iss >> word) ++count;
    return count;
}
```

Обработка исключений в модулях

Python — try/except:

```python
# math_utils.py
def divide(a: float, b: float) -> float:
    """Делит a на b."""
    if b == 0:
        raise ValueError("Деление на ноль")
    return a / b

# main.py
from math_utils import divide

try:
    result = divide(10, 0)
except ValueError as e:
    print(f"Ошибка: {e}")
```

Сравнение:

Python	C++
try/except	try/catch
raise	throw
Exception	std::exception
ValueError	std::invalid_argument

C++ — try/catch:

```cpp
// math_utils.cpp
#include <stdexcept>

double divide(double a, double b) {
    if (b == 0) {
        throw std::invalid_argument("Деление на ноль");
    }
    return a / b;
}

// main.cpp
try {
    double result = divide(10, 0);
} catch (const std::invalid_argument& e) {
    std::cout << "Ошибка: " << e.what() << "\n";
}
```

Собственные исключения в Python:

```python
class MathError(Exception):
    """Базовое исключение для математических операций."""
    pass

class DivisionByZeroError(MathError):
    """Деление на ноль."""
    pass

def divide(a, b):
    if b == 0:
        raise DivisionByZeroError("Деление на ноль")
    return a / b
```

Собственные исключения в C++:

```cpp
#include <stdexcept>

class MathError : public std::runtime_error {
public:
    MathError(const std::string& msg) : std::runtime_error(msg) {}
};

class DivisionByZeroError : public MathError {
public:
    DivisionByZeroError() : MathError("Деление на ноль") {}
};
```

Практика

Задание 1: Функция проверки простоты числа

Задача: Написать программу с вызовом функции проверки простоты числа.

```python
def is_prime(n: int) -> bool:
    """Проверяет, является ли число n простым."""
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

for num in range(1, 20):
    if is_prime(num):
        print(num, end=" ")
# 2 3 5 7 11 13 17 19
```

```cpp
#include <iostream>
#include <cmath>

bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i <= std::sqrt(n); ++i) {
        if (n % i == 0) return false;
    }
    return true;
}

int main() {
    for (int num = 1; num < 20; ++num) {
        if (isPrime(num)) {
            std::cout << num << " ";
        }
    }
    std::cout << "\n";
    return 0;
}
```

Задание 2: Рекурсивный факториал

Задача: Написать программу вычисления факториала через рекурсию.

```python
def factorial(n: int) -> int:
    """Вычисляет факториал числа n рекурсивно."""
    if n <= 1:
        return 1
    return n * factorial(n - 1)

def factorial_iter(n: int) -> int:
    """Итеративная версия."""
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print(factorial(5))       # 120
print(factorial_iter(5))  # 120
```

```cpp
#include <iostream>

long long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

long long factorialIter(int n) {
    long long result = 1;
    for (int i = 1; i <= n; ++i) {
        result *= i;
    }
    return result;
}

int main() {
    std::cout << factorial(5) << "\n";      // 120
    std::cout << factorialIter(5) << "\n";  // 120
    return 0;
}
```

Задание 3: Модуль математических функций



Задача: Реализовать математические функции через модуль.

Python (math_utils.py):

```python
"""Модуль математических функций."""

def gcd(a: int, b: int) -> int:
    """НОД двух чисел (алгоритм Евклида)."""
    while b != 0:
        a, b = b, a % b
    return a

def lcm(a: int, b: int) -> int:
    """НОК двух чисел."""
    return abs(a * b) // gcd(a, b)

def is_perfect(n: int) -> bool:
    """Проверяет, является ли число совершенным."""
    if n < 2:
        return False
    divisors_sum = sum(i for i in range(1, n) if n % i == 0)
    return divisors_sum == n
```

Python (main.py):

```python
from math_utils import gcd, lcm, is_perfect

print(gcd(48, 18))       # 6
print(lcm(4, 6))         # 12
print(is_perfect(28))    # True
```

C++ (math_utils.h):

```cpp
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

int gcd(int a, int b);
int lcm(int a, int b);
bool isPerfect(int n);

#endif
```

C++ (math_utils.cpp):

```cpp
#include "math_utils.h"

int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

bool isPerfect(int n) {
    if (n < 2) return false;
    int sum = 0;
    for (int i = 1; i < n; ++i) {
        if (n % i == 0) sum += i;
    }
    return sum == n;
}
```

Задание 4: Рекурсивные алгоритмы

Задача: Реализовать рекурсивные функции:

- Сумма цифр числа
- Переворот строки
- Проверка палиндрома
- Бинарный поиск

```python
def sum_digits(n: int) -> int:
    """Сумма цифр числа."""
    if n < 10:
        return n
    return n % 10 + sum_digits(n // 10)

def reverse_string(s: str) -> str:
    """Переворот строки."""
    if len(s) <= 1:
        return s
    return reverse_string(s[1:]) + s[0]

def is_palindrome(s: str) -> bool:
    """Проверка палиндрома."""
    if len(s) <= 1:
        return True
    if s[0] != s[-1]:
        return False
    return is_palindrome(s[1:-1])

def binary_search(arr, target, low=0, high=None):
    """Бинарный поиск."""
    if high is None:
        high = len(arr) - 1
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, high)
    else:
        return binary_search(arr, target, low, mid - 1)

# Тесты
print(sum_digits(12345))          # 15
print(reverse_string("Python"))   # nohtyP
print(is_palindrome("racecar"))   # True
print(binary_search([1,2,3,4,5], 3))  # 2
```

```cpp
#include <iostream>
#include <string>

int sumDigits(int n) {
    if (n < 10) return n;
    return n % 10 + sumDigits(n / 10);
}

std::string reverseString(const std::string& s) {
    if (s.length() <= 1) return s;
    return reverseString(s.substr(1)) + s[0];
}

bool isPalindrome(const std::string& s) {
    if (s.length() <= 1) return true;
    if (s[0] != s.back()) return false;
    return isPalindrome(s.substr(1, s.length() - 2));
}

int binarySearch(int arr[], int target, int low, int high) {
    if (low > high) return -1;
    int mid = (low + high) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) return binarySearch(arr, target, mid + 1, high);
    return binarySearch(arr, target, low, mid - 1);
}

int main() {
    std::cout << sumDigits(12345) << "\n";        // 15
    std::cout << reverseString("Python") << "\n"; // nohtyP
    std::cout << isPalindrome("racecar") << "\n"; // 1

    int arr[] = {1, 2, 3, 4, 5};
    std::cout << binarySearch(arr, 3, 0, 4) << "\n";  // 2

    return 0;
}
```

Задание 5. Работа в мини-группах

Формат работы: группы по 3-4 человека.
Задача: Каждая группа получает свой вариант. Необходимо реализовать программу на **двух языках** (Python и C++) и сравнить результаты.

Варианты задач для групп:

Вариант	Задача
1	Модуль для работы со строками: палиндром, анаграмма, подсчёт слов
2	Модуль для работы с числами: простота, совершенство, разложение на множители
3	Рекурсивные алгоритмы: Ханойские башни, перебор подмножеств
4	Модуль для работы с массивами: поиск, сортировка, фильтрация
5	Рекурсивный обход дерева (симуляция на списках)
6	Модуль с обработкой исключений: безопасный ввод, логирование

Критерии оценки:
- Корректность решения на двух языках
- Понимание различий между языками
- Обработка краевых случаев
- Качество форматирования вывода

Презентация результатов: 2–3 минуты на группу.

Требования к отчёту и критерии оценки

Структура отчёта:

1. Титульный лист
2. Цель работы
3. Выполнение заданий:
   - Листинг кода на двух языках
   - Скриншоты результатов
   - Примеры использования функций, рекурсии, модулей
   - Результаты групповой работы
4. Ответы на контрольные вопросы
5. Вывод

Критерии оценки:

Критерий	Мин. балл	Макс. балл
Самостоятельная проработка теории, ответы на вопросы	1	2
Корректность выполнения заданий, рабочий код	3	6
Качество кода (PEP8, структура, комментарии)	1	2
ИТОГО	**5**	**10**

Формат сдачи: отчёт в формате PDF или DOCX + код в Git-репозитории (не менее 3 коммитов).

Контрольные вопросы

Контрольные вопросы

Вопросы для самопроверки:

По функциям:

1. Что такое функция? Зачем нужны функции?
2. Как объявить и вызвать функцию в Python и C++?
3. В чём разница между передачей по значению и по ссылке?
4. Что такое области видимости? Правило LEGB?
5. Что такое перегрузка функций? В каком языке она есть?
6. Что такое лямбда-выражение?

По рекурсии:

7. Что такое рекурсия? Обязательные компоненты?
8. В чём разница между рекурсией и итерацией?
9. Когда использовать рекурсию?

По модулям:

10. Что такое модуль? Зачем нужны модули?
11. Как создать модуль в Python? А в C++?
12. Что такое if __name__ == "__main__"?
13. Как обрабатывать исключения в модулях?



Ответы должны быть в отчёте.

Задания для самостоятельной работы

Задания для самостоятельной работы

Базовый уровень (обязательно для всех):

Выполняется на Python и\или C++, код загружается в Git-репозиторий.

1. Написать модуль geometry.py / geometry.cpp с функциями вычисления площади круга, прямоугольника и треугольника. Использовать параметры по умолчанию (для круга — точность pi).
2. Написать рекурсивную функцию возведения числа в степень (power(base, exp)). Сравнить её производительность с итеративной версией и встроенной функцией ** / std::pow.
3. Вынести функции из предыдущих заданий в отдельные файлы. В основной программе (main.py / main.cpp) реализовать меню для выбора и вызова этих функций.

Продвинутый уровень (для дополнительных баллов / по желанию):

4. **Мемоизация**: Реализовать рекурсивную функцию Фибоначчи с кэшированием результатов (в Python — через словарь или декоратор @lru_cache, в C++ — через std::map или std::unordered_map).
5. **Лямбда-выражения**: Используя лямбда-выражения, реализовать сортировку списка/вектора строк по длине, а затем по алфавиту (при равенстве длин).
6. **Обработка ошибок**: Дописать модуль из задания №1 так, чтобы он выбрасывал исключения при некорректных входных данных (например, отрицательный радиус). Обработать эти исключения в главном меню.



Требования к оформлению СР:

- Наличие docstring (Python) или комментариев (C++) для каждой функции.
- Код должен быть отформатирован (black для Python, clang-format для C++).

Спасибо
за внимание

